

JFEM Dual Back-End Strategy

Common Nastran front end, Nastran-parity back end, and TACS-formulation back end

JFEM development documentation

Generated 2026-06-01

Purpose of This Deck

Main objective

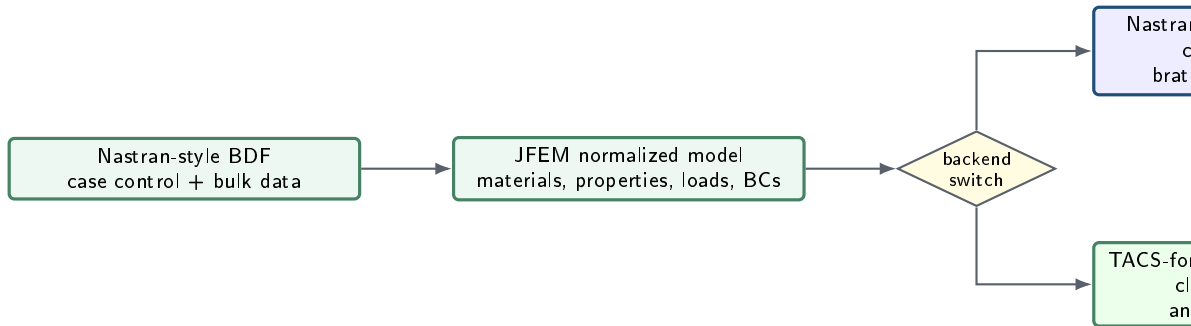
Define a solver architecture where the same JFEM input workflow can run with either:

- a **Nastran-parity** formulation for drop-in agreement, or
- a **TACS-formulation** path for clean adjoints, AD, sensitivities, and optimization.

Important constraint

The goal is not to clone TACS and not to add a Python or interactive interface. The goal is to absorb the useful formulation ideas and expose them through JFEM's existing batch solver.

Strategic Picture



The front end and output layer stay common. Only the element/residual/sensitivity back end changes.

What We Mean by Front End and Back End

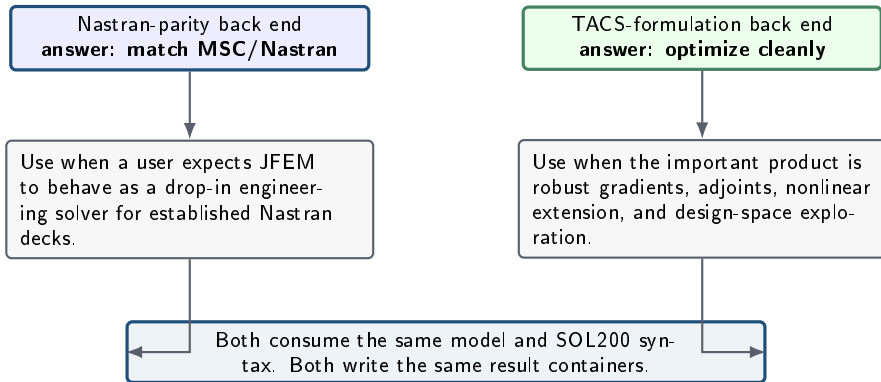
Common front end

- BDF reader and INCLUDE resolution.
- Case control normalization.
- Card extraction and model building.
- Coordinate systems, properties, materials.
- SOL101, SOL103, SOL105, SOL106, SOL200 routing.
- Manifest and batch execution.

Back end

- Element residuals and tangents.
- Stiffness, mass, geometric stiffness.
- Stress/strain recovery.
- Response partials.
- Adjoint equations.
- Design sensitivities.

Why Two Back Ends?



Nastran, TACS, and JFEM: Roles

Aspect	MSC/Nastran	TACS	JFEM target
Primary identity	Industrial finite element solver	Open optimization-oriented FE framework	Nastran-compatible solver with optional clean formulation back end
Input semantics	BDF and case control reference behavior	BDF support through py-TACS, but not the main standard	Common JFEM BDF front end
Formulation visibility	Proprietary internals	Open C++ implementation	Julia implementation with transparent kernels
Best strength	Broad production robustness	Adjoint, sensitivities, KS aggregation, optimization architecture	Combine Nastran-style workflow with transparent differentiable kernels
Main limitation for us	Internal notes unavailable	Not a Nastran drop-in clone and Python-centric tooling	Must preserve parity while adding a second formulation path

Current Codebase Baseline

43

Julia source files

42,162

Julia source lines

6,968

lines in `assembly.jl`

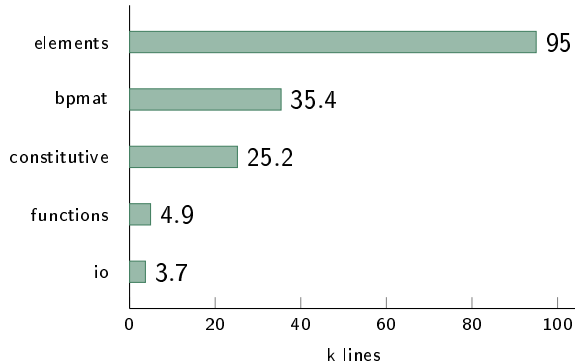
Largest JFEM modules	Lines
<code>solver/assembly.jl</code>	6,968
<code>FEMKernels.jl</code>	5,796
<code>solver/solve_case.jl</code>	3,830
<code>Export.jl</code>	3,036
<code>solver/optimize_thickness.jl</code>	2,829
<code>solver/adjoint_buckling.jl</code>	2,442
<code>JFEMSolver.jl</code>	2,002

Implication

The first architectural task is not adding many features. It is separating shared solver orchestration from formulation-specific kernels.

TACS Source Areas Useful to JFEM

TACS area	Files	Lines
elements	157	94,957
bpmat	47	35,434
constitutive	54	25,171
functions	28	4,915
io	10	3,706
TACSAssembler.cpp	1	5,227
TACSBuckling.cpp	1	853



TACS Ideas to Import, Not TACS Infrastructure

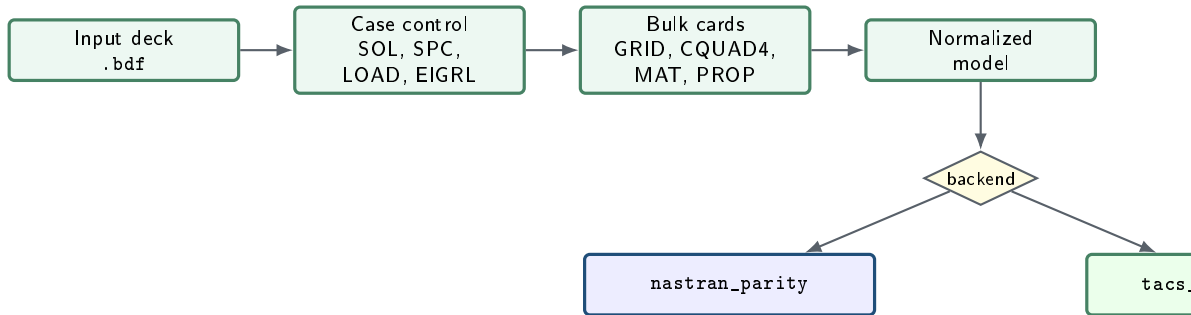
Import the ideas

- Residual-first element formulation.
- Consistent tangent matrices.
- Clean constitutive objects.
- KS aggregation for constraints.
- Adjoint response gradients.
- Buckling eigenvalue sensitivities.

Do not import

- Python user interface.
- Interactive workflows.
- pyTACS BDF layer.
- TACS build system.
- A full C++ wrapper.
- TACS as a runtime dependency.

Common Front End: Preserved User Contract



The user should not need different input files for different back ends.

Back-End Selection Mechanisms

Command-line and environment

```
# Default: Nastran-parity path
julia --project=. src/main.jl model.bdf output

# Explicit clean formulation path
set JFEM_BACKEND=tacs_formulation
julia --project=. src/main.jl model.bdf output
```

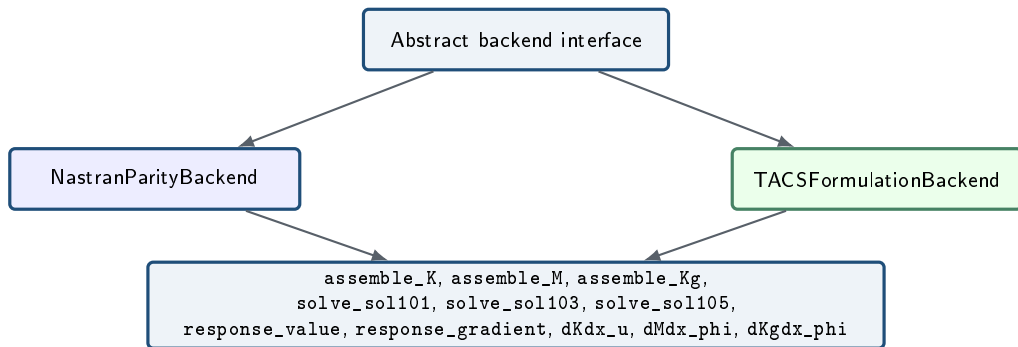
Manifest/batch execution

```
{
  "backend": "tacs_formulation",
  "cases": [
    {"input": "case_001.bdf",
     "output_dir": "runs/case_001"}
  ]
}
```

Default rule

The default must remain `nastran_parity` until the TACS-formulation path has its own validation envelope.

Backend Interface Contract



Proposed Julia Types

```
abstract type AbstractJFEMBackend end

struct NastranParityBackend <: AbstractJFEMBackend end
struct TACSFormulationBackend <: AbstractJFEMBackend end

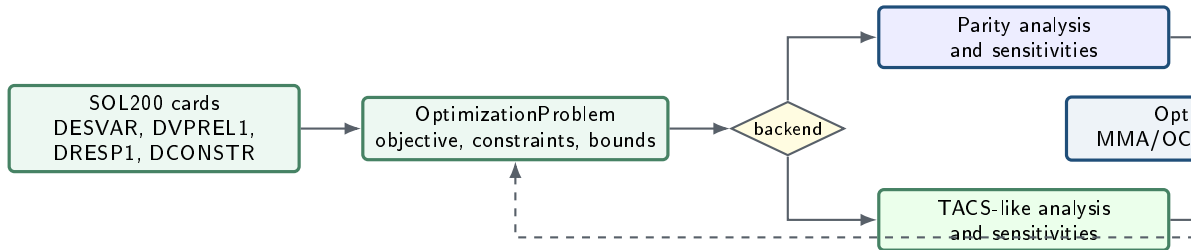
backend_from_model(model) =
    get(model, "backend", get(ENV, "JFEM_BACKEND", "nastran_parity"))

solve_model(model) = solve_model(backend_from_model(model), model)
```

Design goal

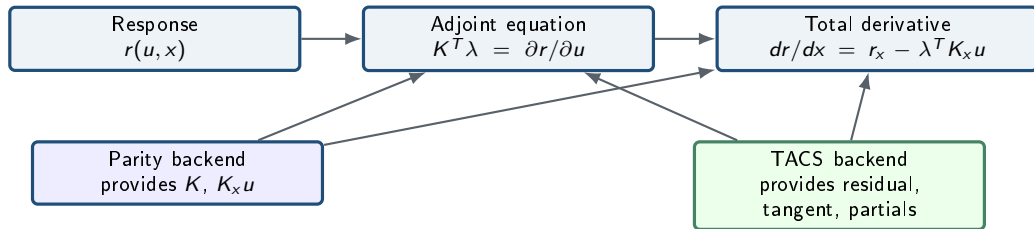
Current solver behavior is preserved by implementing the existing path as `NastranParityBackend` first, before any TACS-formulation code is activated.

SOL200 Should Sit Above the Backend



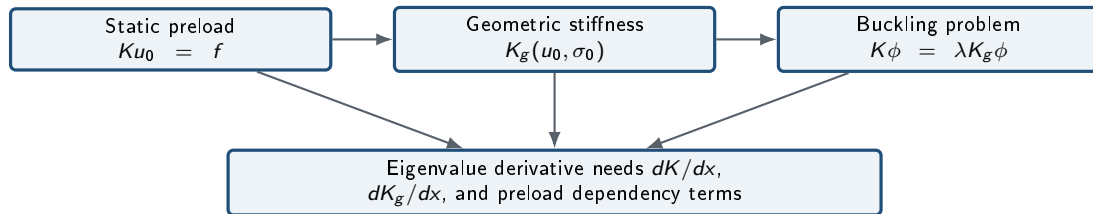
The SOL200 syntax describes the design problem. The backend supplies the physics and gradients.

Shared Sensitivity and Adjoint Layer



The algebra is common; only the operator evaluation differs.

SOL105 Sensitivity Contract



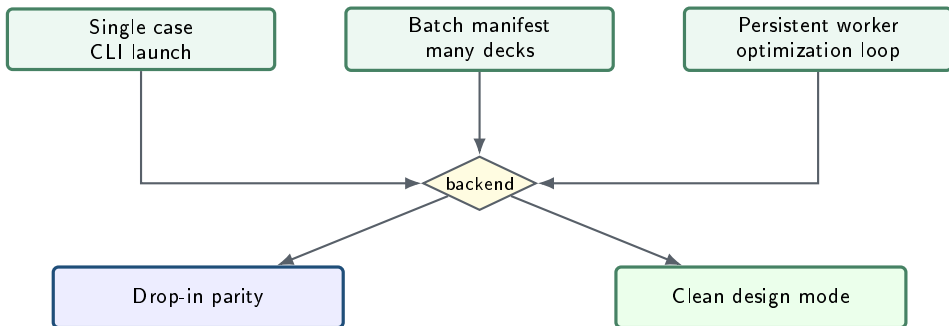
Practical rule

SOL105 parity must not regress while the TACS path is developed. The parity backend remains the guarded production path.

Comparison: Nastran-Parity Backend vs TACS-Formulation Backend

Dimension	Nastran-parity back end	TACS-formulation back end
Goal	Reproduce Nastran trends and validation envelopes	Provide clean, consistent residuals and derivatives
Primary risk	Accumulated calibration switches and legacy branches	Initial mismatch with Nastran for some cases
Default status	Default production mode	Opt-in formulation mode
Element style	Existing JFEM kernels, parity-specific options	Residual-first kernels inspired by TACS architecture
Adjoint path	Existing static/buckling adjoint, routed through interface	Same adjoint layer, backend-specific partials
Optimization	SOL200-lite and sizing routes	Same SOL200 route after interface is stable
Validation target	MSC/Nastran comparison	Internal consistency, FD/complex-step checks, then Nastran comparison

Expected User-Facing Modes



The same back-end switch should work for interactive development, command-line execution, batch runs, and Python-triggered optimization loops.

TACS-Inspired Element Roadmap

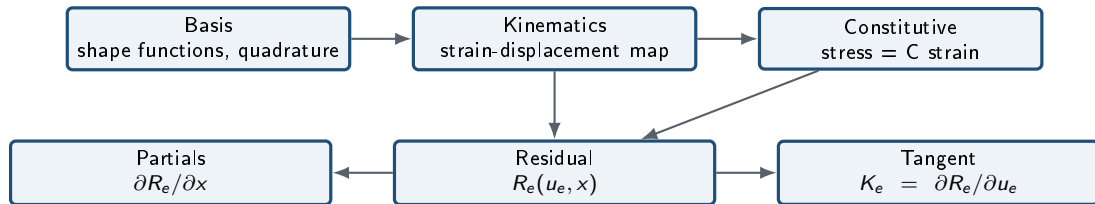
Vertical slice 1

- CQUAD4
- PSHELL
- MAT1
- SOL101
- Compliance and displacement responses
- Thickness sensitivity

Vertical slice 2

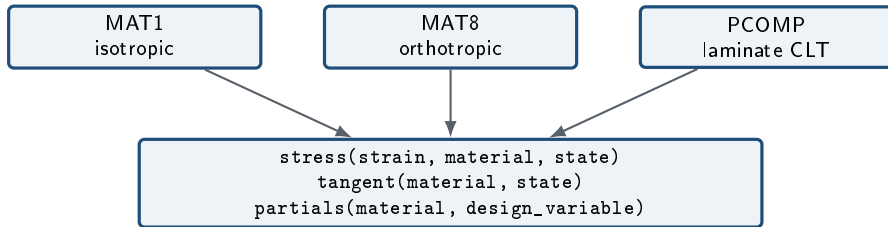
- PCOMP
- MAT8
- Laminate constitutive derivatives
- SOL105 preload and buckling
- Buckling eigenvalue sensitivity
- KS stress aggregation

Element Kernel Shape



Residual-first kernels are easier to test, easier to differentiate, and cleaner for SOL106/SOL200 growth.

Constitutive Object Roadmap



Why this matters

TACS obtains much of its optimization strength from clean constitutive abstractions. JFEM should adopt that pattern in Julia without adopting TACS runtime infrastructure.

Use AD where it is strong

- Element-level residual partials.
- Scalar response partials.
- Constitutive derivatives.
- Regression checks against analytic derivatives.

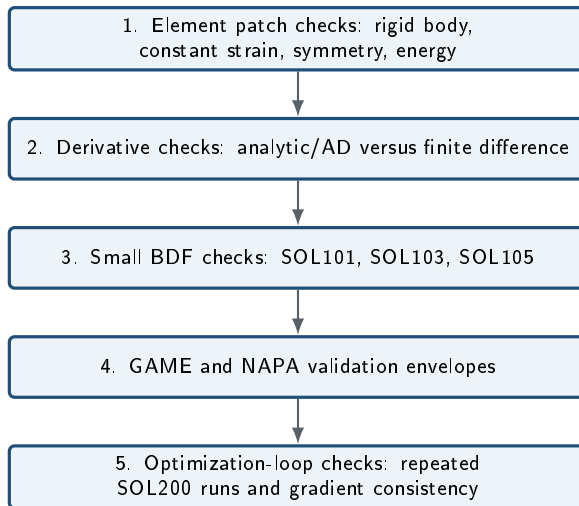
Use analytic formulas where needed

- Large sparse linear systems.
- Eigenvalue sensitivity equations.
- Laminate CLT derivatives.
- Mass and volume derivatives.

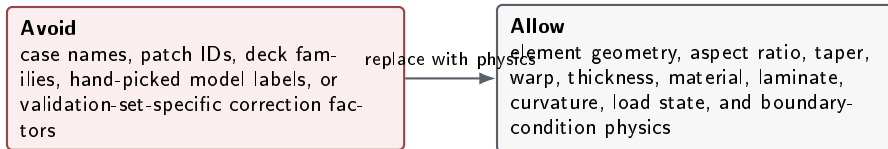
Guiding rule

AD should support the formulation. It should not replace engineering knowledge of sparse FE adjoints.

Validation Ladder



No Case-Specific Formulation Rules



Reason

A TACS-formulation path must be a formulation, not a calibration table. The Nastran-parity path may keep guarded compatibility switches, but the new path should be physically parameterized.

Implementation Phase 0: Refactor Without Behavior Change

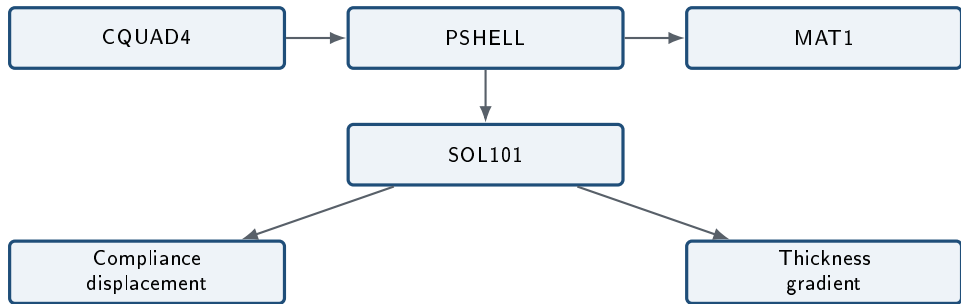
Create interface

- Add backend types.
- Route current solver through `NastranParityBackend`.
- Preserve all current outputs.
- Add backend name to result metadata.

Acceptance

- Existing SOL101/103/105 reports unchanged.
- GAME SOL105 parity does not regress.
- SOL200-lite decks still run.
- Manifest and worker paths still run.

Implementation Phase 1: First TACS-Formulation Slice



This phase proves the architecture before PCOMP, buckling, and nonlinear work are added.

Implementation Phase 2: Composite and Buckling Capability

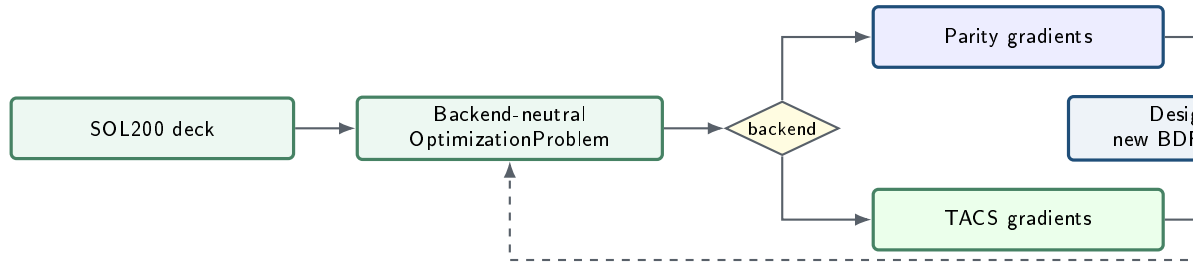
Composite path

- MAT8 constitutive.
- PCOMP laminate object.
- Ply thickness and angle derivatives.
- Laminate mass derivatives.

Buckling path

- Static preload state object.
- Consistent geometric stiffness.
- Eigenvalue response object.
- Buckling adjoint derivative check.

Implementation Phase 3: SOL200 Unification



Backend-Specific Result Metadata

```
results = Dict(  
  "sol_type" => 105,  
  "backend" => "tacs_formulation",  
  "backend_version" => "0.1.0",  
  "formulation" => Dict(  
    "shell" => "residual_first_quad4",  
    "constitutive" => "mat1_pshell",  
    "geometric_stiffness" => "consistent_preload"  
  ),  
  "subcases" => subcases,  
  "solver_diagnostics" => diagnostics  
)
```

This allows validation reports to compare both numeric results and formulation provenance.

Risk Register

Risk	Why it matters	Mitigation
SOL105 parity regression	Current users need drop-in behavior	Default remains <code>nastran_parity</code> ; guard cases run before merge
Backend leakage	Common code starts depending on one formulation	Define backend interface and keep tests for both paths
Gradient mismatch	Optimization may converge to wrong answer	Mandatory finite-difference and adjoint residual checks
Too much TACS copying	Project becomes unmaintainable	Import concepts, not C++ infrastructure or Python flow
Duplicated SOL200 logic	Divergent behavior between backends	SOL200 parser and optimizer stay backend-neutral

Testing Matrix

Capability	SOL101	SOL103	SOL105	AD	Adjoint	SOL200
Nastran-parity backend	yes	yes	yes	partial	yes	lite
TACS slice 1	yes	no	no	yes	yes	no
TACS slice 2	yes	yes	yes	yes	yes	partial
TACS production target	yes	yes	yes	yes	yes	shared



Against Nastran

- Compare displacements, reactions, eigenvalues, MAC, stresses.
- Use it as the production engineering reference.
- Accept that internal implementation details are not visible.

Against TACS

- Compare formulation concepts and derivative quality.
- Use open code to guide architecture and equations.
- Do not require identical input or runtime layers.

Expected Development Folder Layout

```
src/backend/  
  BackendInterface.jl  
  nastran_parity/  
    NastranParityBackend.jl  
  tacs_formulation/  
    TACSFormulationBackend.jl  
  basis.jl  
  constitutive.jl  
  shell_quad4.jl  
  residuals.jl  
  sensitivities.jl  
  sol101.jl  
  sol103.jl  
  sol105.jl  
src/optimization/  
  SOL200Problem.jl  
  Responses.jl  
  AdjointCore.jl  
  Optimizers.jl
```

The exact filenames can evolve, but this separation should be preserved.

Acceptance Gates by Phase

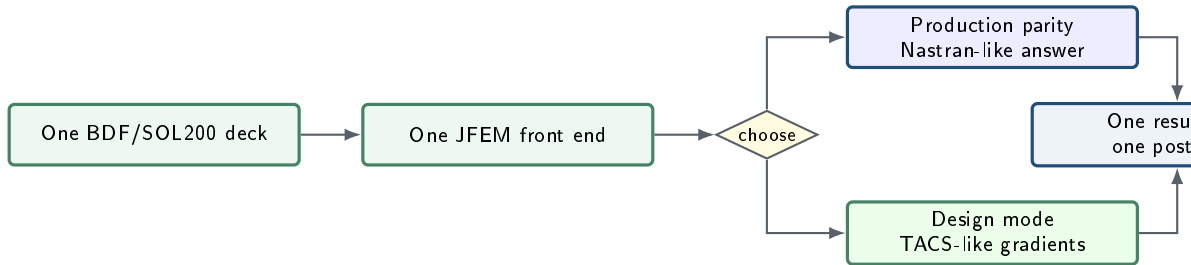
Phase gates

- 1 Current backend wrapped with zero numerical changes.
- 2 TACS SOL101 CQUAD4 slice passes element and derivative checks.
- 3 TACS PCOMP/MAT8 passes laminate derivative checks.
- 4 TACS SOL105 passes small buckling derivative checks.
- 5 Shared SOL200 routes both backends.

Never skip

- Baseline parity campaign.
- Finite-difference sensitivity audit.
- Backend metadata in output.
- No case-name or patch-name rules.

Long-Term Target



Same workflow. Two physics engines. Shared optimization language.

Summary

- JFEM should preserve a common BDF/SOL200 front end and common output layer.
- The current solver should become the explicit `nastran_parity` backend.
- The new `tacs_formulation` backend should implement residual-first kernels, clean constitutives, and robust derivatives.
- Sensitivity, AD, adjoint, and SOL200 logic should be backend-neutral wherever possible.
- The TACS effort should import architecture and formulation ideas, not Python tooling or C++ runtime dependencies.